USS Pampanito — AI-Powered Interactive Audio Docent

Application Overview & Technical Summary

---

1. What Is This?

The USS Pampanito AI Docent is a smartphone-accessible, voice-driven audio tour for the USS Pampanito (SS-383), a World War II fleet submarine and National Historic Landmark preserved at Fisherman's Wharf, San Francisco.

Traditional audio tours are passive — visitors hear a fixed narration and move on. This application turns the tour into a two-way conversation by letting visitors ask questions by voice at any moment and receive spoken, historically accurate answers within seconds, drawn exclusively from verified museum-grade sources.

No app download is required. Visitors open a URL on any smartphone browser — including iPhone Safari. The entire system runs on local hardware aboard or adjacent to the vessel, with no internet dependency during operation.

---

2. The Problem It Solves

Museum visitors are curious. They stand inside a torpedo room and want to know whether sailors actually slept next to live torpedoes, or which bunk was the worst to sleep in, or how many men were assigned to the boat. Fixed audio narration cannot answer these questions; human docents cannot be everywhere at once.

- Curiosity goes unrewarded — visitors leave with unanswered questions.
- Non-English-speaking visitors receive no additional context.
- Staff cannot scale to provide personalised responses to hundreds of visitors daily.

The AI Docent solves these problems by providing an always-available, on-premises AI guide that responds to natural spoken questions in real time.

---

3. Visitor Experience — How It Works

Step	What Happens
1. Arrive at a compartment	The visitor enters a compartment (e.g., the Control Room, Forward Torpedo Room) and taps to hear the narrated audio for that space.
2. Listen to narration	Pre-recorded audio plays — the same narration currently offered by the Pampanito tour.

Step	What Happens
3. Ask a question	At any point, the visitor taps the microphone icon and speaks naturally: "How did the torpedo work?" or "Where did the crew sleep?"
4. AI retrieves an answer	The app sends the transcript to the on-premises AI guide, which searches curated historical sources and assembles a direct spoken answer in under two seconds.
5. Answer plays aloud	The answer is read aloud through the visitor's phone speaker or headphones, with a spoken location prefix when relevant (e.g., "In the Crews Mess…").
6. Follow-ups offered	The guide suggests related questions to keep exploration going.
7. Unanswered questions	If the system cannot find a reliable answer, it says so honestly and routes the question to museum staff for follow-up.

4. Knowledge Sources

The AI guide answers exclusively from three curated, authoritative corpora. It does not search the internet and cannot fabricate information.

Source	Size	Description
Pampanito Tour Script	213 chunks	The official compartment-by-compartment narration for all 11 interior compartments plus forward and after decks. Each chunk is tagged with a compartment ID, location context, and display citation.
DieselSubs FAQ Archive (dieselsubs.com)	229 chunks	A comprehensive technical and historical reference on WW2 diesel-electric submarines — crew life, weapons, engineering, tactics. Structured as Q&A pairs for precision retrieval.
DieselSubs Shorts	31 chunks	Concise background explanations on submarine systems, operations, and history. Used to supplement answers when the other sources are not sufficient.

Every answer includes a citation identifying which source and chunk the information came from.

5. Key Features

Feature	Detail
Voice input	Visitor speaks naturally — no typing. Speech recognition via the browser's built-in Web Speech API (no cloud service required).
Spoken answers	Text-to-speech delivers answers aloud — eyes-free while exploring the boat.

Feature	Detail
Multilingual audio	Answers available in English, Spanish, French, German, Japanese, and Chinese.
Location-aware retrieval	The AI knows which compartment the visitor is standing in and weights results accordingly.
No internet required	Fully self-contained — runs on a laptop or small server on or near the vessel.
No app download	Progressive Web App served over HTTPS — works in any modern smartphone browser.
Honest fallback	If no reliable answer is found, a recorded message says so and routes the question to staff.
Pre-cached FAQ audio	Frequently asked questions have pre-recorded TTS answers that play instantly.
Suggested follow-ups	The guide proposes related questions to deepen visitor engagement.
Unanswered question log	All unanswered questions are captured and e-mailed to the museum historian for content development.

6. Technical Architecture

The application is built on three layers: a browser-based frontend, an on-premises Python API server, and a local retrieval engine backed by curated JSONL corpora.

6.1 Frontend (`web/tour.html`)

A single-page application served directly by the backend. Designed for iOS Safari on iPhone — the primary visitor device.

- **Compartment-aware navigation** — 11 compartments + fore/aft deck, each with a unique ID passed to the API.
- **AudioContext playback** — GainNode normalises volume across narration, TTS answers, and pre-recorded fallback audio.
- **Voice input** — Browser Web Speech API (`webkitSpeechRecognition`) captures the visitor's question directly in the browser; no audio upload required.
- **Answer pipeline** — transcript POSTed to `/ask`; answer text POSTed to `/tts` for speech synthesis; audio streamed and played immediately.
- **No-cache headers** — visitors always load the latest tour version without a hard refresh.

6.2 API Server (`api/main.py`)

FastAPI application hosted with Uvicorn behind a self-signed TLS certificate (required for microphone access on mobile browsers).

Endpoint	Method	Purpose
`POST /ask`	JSON: `question_text`, `compartment_id`	Main Q&A endpoint. Runs retrieval + extractive synthesis; returns a structured answer with citations and `partial_match` flag.
`POST /tts`	JSON: `text`, `language`	Converts answer text to speech via OpenAI TTS; returns an MP3 audio stream.
`POST /contact`	Multipart form	Captures visitor contact info and unanswered question; e-mails the historian.
`GET /web/*`	Static files	Serves the tour HTML, audio files, and assets.

6.3 Retrieval Engine

A custom token-overlap retrieval system with vocabulary expansion — no vector database required. Designed for high accuracy on a closed, domain-specific corpus.

- **Tokenisation & stopword filtering** — query and corpus text are lowercased, punctuation-stripped, and filtered against a domain-tuned stopword list (including universal noise words like "submarine" that appear in every chunk).
- **Synonym expansion (`QUERY_SYNONYMS`)** — query tokens are expanded with domain synonyms before scoring. For example, "eat" expands to ate, galley, mess, food, meal; "men" expands to crew, sailors, crewmen, complement. This bridges the vocabulary gap between visitor phrasing and corpus language.
- **Weighted corpus scoring** — Tour = 3.0×, FAQ = 1.2×, Shorts = 0.8×. Tour chunks in the visitor's current compartment naturally score highest.
- **FAQ title-match bonus** — FAQ chunks whose question title covers all synonym-expanded query tokens receive a 4× coverage bonus, ensuring the most precisely targeted FAQ answer wins over tangentially related tour passages.
- **Quantity boost** — for "how many" questions, chunks containing number words receive a 1.5× score multiplier.
- **Intent detection** — detects WHERE questions (prepends "In the [location]." to the answer), HOW MANY questions (activates quantity checking), and Mark 14/18 comparison questions (activates comparison-language boost).

6.4 Extractive Answer Synthesis

Answers are assembled by extracting the most relevant sentences from the top-ranked corpus chunk — no LLM generation in the default deployment path. This guarantees factual fidelity and eliminates hallucination risk.

- Sentences are filtered and ranked by synonym-expanded term overlap with the query.
- A second chunk is consulted if the first does not yield a complete answer.
- Speech filler words ("uh", "um", "er") are automatically stripped from oral-history transcript chunks before the answer is read aloud.
- A `partial_match` flag is returned when the answer is topically relevant but does not directly answer the question. The frontend plays a recorded "I don't have a direct answer, but here's what I know"

prefix before reading the answer.

---

7. Answer Quality & Partial Match Logic

A key reliability feature is the system's ability to distinguish between a direct answer and a relevant-but-indirect answer. The `partial_match` flag is set `True` in any of the following cases:

Condition	Example
None of the core query terms appear in the assembled answer	Question about "escape hatches" but answer discusses hull construction
"How many X" question but no sentence contains both X and a number	"How many torpedoes" but answer only says they were stored forward
Question contains a superlative (worst, best, hardest…) but the answer does not address the judgment	"What was the worst bunk to sleep in" — answer describes bunks but cannot rank them

When `partial_match` is `True`, the frontend plays a soft spoken prefix (`nodirectanswer.mp3`) before reading the answer, setting visitor expectations correctly.

If no relevant content is found at all (zero hits), a separate "nothing found" audio clip plays and the question is queued for historian review.

---

8. Unanswered Questions Workflow

Every question the system cannot answer is automatically captured and forwarded to the museum historian via e-mail, creating a continuous content improvement loop:

1. Visitor asks a question the system cannot answer.
2. The system plays the graceful fallback audio and optionally collects the visitor's contact information.
3. The question and visitor details are e-mailed to the historian's inbox.
4. The historian evaluates the question; if valid, new content is added to the corpus or the website.

This loop ensures the knowledge base grows over time, driven directly by actual visitor curiosity.

---

9. Deployment

Component	Detail
Hardware	MacBook Pro or equivalent — can run on a Mac Mini or small server. No GPU required.
Network	Local Wi-Fi access point (no internet uplink needed during operation). Visitors connect via QR code.
TLS	Self-signed certificate on port 8443 — required by browsers for microphone access (HTTPS).

Component	Detail
Python stack	Python 3.10+, FastAPI, Uvicorn, OpenAI SDK (TTS only). All dependencies in a local virtual environment.
Server startup	Single command: `bash start_https.sh` — kills any prior instance, sets environment variables, starts Uvicorn.
LLM flag	`USE_LLM=false` (default) — fully local, no OpenAI API calls for Q&A. Set to `true` to enable GPT-based synthesis fallback when funded.

---

10. Current Status & Vision

The application is a fully functional demonstration prototype. It covers all 11 interior compartments plus the fore and aft decks of the Pampanito, and the AI guide can answer hundreds of historical and technical questions about the ship and her crew.

Current capabilities:

- Voice Q&A in 6 languages across all compartments
- Sub-2-second response time on local hardware
- Intelligent fallback with honest partial-answer signalling
- Continuous improvement loop via historian e-mail workflow
- Zero internet dependency during visitor operation

Near-term roadmap:

- LLM-backed synthesis (GPT-4o) for richer, more conversational answers when funding is secured
- Pre-cached TTS audio for the 50 most common questions — instant playback without any API latency
- Expanded corpus coverage: crew oral histories, patrol logs, and post-war interviews
- QR code self-check-in for analytics (compartment traffic, popular questions)

The long-term vision is a tour where every visitor — regardless of language, prior knowledge, or pace — can have a personal, informed conversation with the history they are standing inside. The submarine becomes a place where curiosity is always rewarded.

USS Pampanito — SS-383 | Fisherman's Wharf, San Francisco | AI Docent Prototype | 2026